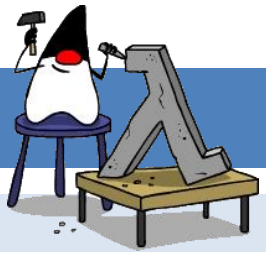
GROEIPROJECT
MODULE 4: LAMBDA'S & STREAMS



1. VOORBEREIDING

1.1. Open het groeiproject en maak daarin een vierde module: `m4_streams`

1.2. Pas de `build.gradle` aan.

1.3. Kopieer van module `m1_herhaling` de hele `be.kdg...` map (met o.a. de oorspronkelijke basis- en multiklasse en de dataklasse) naar de nieuwe module `m4_streams`

2. LAMBDA EXPRESSIONS EN METHOD REFERENCES

2.1. In Module `m1_herhaling` van het groeiproject schreef je voor de multiklasse verschillende sorteermethoden.
Vervang die nu in deze module in jouw multiklasse door één algemene sorteermethode met de naam **`sortedBy`**, die als parameter een `Function` heeft, die het sorteercriterium uit je basisklasse bepaalt.
Te implementeren methode in onze voorbeeld-multiklasse `Dictators`:

```
public List<Dictator> sortedBy(Function<Dictator, Comparable> function)
```

2.2. Maak een nieuwe main klasse `Demo_4` en test de nieuwe methode **drie keer** uit met telkens een ander sorteercriterium. Geef de Function door als een **method reference**. Bijvoorbeeld:

```
System.out.println("\nDictators gesorteerd op naam:");
for (Dictator dictator : dictators.sortedBy(Dictator::getName)) {
    System.out.println(dictator);
}
```

2.3. Maak een nieuwe package `util` met daarin een klasse `XxxFunctions` (in ons voorbeeld: `DictatorFunctions`) met daarin een aantal generieke **static** methoden.

Van de eerste methode `filteredList` ziet de header er als volgt uit:

```
public static <T> List<T> filteredList(List<T> dictatorList, Predicate<T> predicate)
```

a. Merk op hoe gebruik wordt gemaakt van generics.
b. Als eerste parameter komt een List van basisobjecten binnen (in ons voorbeeld: een List van Dictators)
c. Als tweede parameter een `Predicate`-functie die zal bepalen op welke conditie gefilterd zal worden
d. Maak voorlopig gebruik van een for-loop
e. De returnwaarde is een gefilterde List van basisobjecten.

2.4. In de klasse `Demo_4`:

a. Vraag aan de klasse `Data` een gevulde List met basisobjecten.
b. Roep drie keer de zopas geschreven filtermethode op en geef telkens een ander `Predicate` mee.
c. Geef het `Predicate` bij de oproep mee als een Lambda-expression.
d. Controleer de afdruk.

Voorbeeld van oproep:

```
dictatorList = DictatorFunctions.filteredList(dictatorList,
                  dictator -> dictator.getVictims() > 3);
```

Mogelijke afdruk:

```
Toepassing 3 keer FilteredList met telkens een ander Predicate:
Alle mannelijke dictators met een stalinistisch regime die meer dan 3 mln slachtoffers
maakten:
Jozef Stalin           (°1878) Rusland         regime: Stalinisme         45,0 mln doden
Kim Il Sung            (°1912) Noord-Korea     regime: Stalinisme          4,0 mln doden
```

2.5. Maak in de klasse `XxxFunctions` een tweede methode `average` met de volgende signature:

```
public static <T> Double average (List<T> dictatorList, ToDoubleFunction<T> mapper)
```

a. Als eerste parameter komt een List van basisobjecten binnen (in ons voorbeeld: `dictatorList`)
b. Als tweede parameter een `Function` die zal bepalen op welk aspect het gemiddelde zal berekend worden
c. Maak voorlopig gebruik van een for-loop
d. De returnwaarde is het gemiddelde.

2.6. In de klasse `Demo_4` ga je deze methode uittesten:

a. Vraag aan de klasse `Data` opnieuw een vers gevulde List met basisobjecten.
b. Roep twee keer de zopas geschreven methode `average` op en geef telkens een andere `Function` mee, die bepaalt van welk attribuut het gemiddelde moet berekend worden.
c. Geef de `Function` bij de oproep mee als een Lambda-expression.

Voorbeeld van oproep:

```
System.out.printf("Gemiddeld aantal slachtoffers: %.1f mln\n",
     DictatorFunctions.average (dictatorList, Dictator::getVictims));
```

2.7. Maak in de klasse `XxxFunctions` een derde methode `countIf` met de volgende signature:

```
public static <T> long countIf(List<T> dictatorList, Predicate<T> predicate)
```

a. Als eerste parameter komt een List van basisobjecten binnen (in ons voorbeeld: `dictatorList`)
b. Als tweede parameter een `Predicate` dat bepaalt op welk aspect geselecteerd wordt
c. Maak voorlopig gebruik van een for-loop
d. De returnwaarde is het aantal objecten dat aan het predicate voldoet.

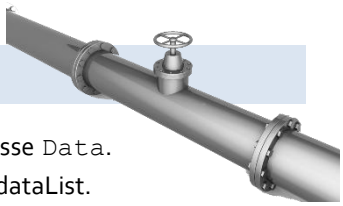
2.8. In de klasse `Demo_4` ga je deze methode uittesten:

a. Roep twee keer de zopas geschreven methode op en geef telkens een ander `Predicate` mee.
b. Geef het `Predicate` bij de oproep mee als een Lambda-expression.

Voorbeeld van oproep:

```
System.out.printf("Aantal dicators met wreedheid > 50%%: %d\n",
        DictatorFunctions.countIf(dictatorList, dictator -> dictator.getCruelty() > 50.0));
```

3. STREAMS



In de klasse `Demo_4` vraag je opnieuw een verse List van basisobjecten op aan de klasse `Data`.
Je gaat nu via streaming de onderstaande opdrachten uitvoeren, toegepast op jouw dataList.
Programmeer alles zo compact en efficiënt mogelijk!

1.1. **Tel** hoeveel elementen aan een bepaald datum-criterium beantwoorden.
Voorbeeld van afdruk:

```
Aantal dictators dat na 1900 geboren is: 9
```

1.2. **Sorteer** (op 2 attributen!) en druk af
Voorbeeld van afdruk:

```
Alle dictators gesorteerd op regime en vervolgens op naam:
Mao Tse-tung           (°1893) China          regime: Communisme             8,0 mln doden
Nicolae Ceausescu      (°1918) Roemenië       regime: Despotisme             0,5 mln doden
Laurent-Désiré Kabila  (°1939) Congo          regime: Dictatuur              0,3 mln doden
Benito Mussolini       (°1883) Italië         regime: Fascisme               2,0 mln doden
Ayatollah Khomeini     (°1902) Iran           regime: Fundamentalisme        8,5 mln doden
enz…
```

1.3. Zet een attribuut om naar hoofdletters, verwijder de dubbels, sorteer in aflopende volgorde en zet de elementen in één string, gescheiden door komma's.
Voorbeeld van afdruk:

```
Alle dictatorlanden in hoofdletters, omgekeerd gesorteerd en zonder dubbels:
SPANJE, RUSLAND, ROEMENIË, OEGANDA, NOORD-KOREA, ITALIË, IRAN, FILIPIJNEN, DUITSLAND,
CONGO, CHINA, CHILI, CAMBODJA
```

1.4. **`filter`** op een bepaald criterium en pik er eentje uit met **`findAny`**
Voorbeeld van afdruk (opgelet: dat is altijd dezelfde; weet je waarom? Lees de FAQ op Canvas):

```
Een willekeurige dictator met meer dan 75% wreedheid:
Laurent-Desire Kabila : 82.0
```

1.5. Toon twee **kampioenen** in een specifiek gebied
Voorbeeld van afdruk:

```
De kampioen in aantal slachtoffers: Adolf Hitler
De kampioen in regeerperiode: Kim II Sung
```

1.6. Filter op een String-criterium, **map** elk element op een String (bijvoorbeeld op naam), sorteer alfabetisch en collect als een List van String. Voorbeeld van afdruk:

```
List met gesorteerde dictatornamen die beginnen met 'A':
[Adolf Hitler, Atilla de Hun, Augusto Pinochet, Ayatollah Khomeini]
```

1.7. Sorteer op een datum-attribuut en **partitioneer** in twee groepen (voor/na die datum)**:**
Voorbeeld van afdruk:

```
Sublist met dictators geboren VOOR 1900:
Atilla de Hun          (° 406) Hunnische Rijk regime: Barbarisme             1,4 mln doden
Jozef Stalin           (°1878) Rusland        regime: Stalinisme            45,0 mln doden
Benito Mussolini       (°1883) Italie         regime: Fascisme               2,0 mln doden
Adolf Hitler           (°1889) Duitsland      regime: Nazisme               66,0 mln doden
Francisco Franco       (°1892) Spanje         regime: Militaire dictatuur    5,0 mln doden

Sublist met dictators geboren NA 1900:
Ayatollah Khomeini     (°1902) Iran           regime: Fundamentalisme        8,5 mln doden
Kim II Sung            (°1912) Noord-Korea    regime: Stalinisme             4,0 mln doden
Augusto Pinochet       (°1915) Chili          regime: Militaire dictatuur    0,1 mln doden
Ferdinand Marcos       (°1917) Filipijnen     regime: Kleptocratie           0,1 mln doden
Nicolae Ceausescu      (°1918) Roemenië       regime: Despotisme             0,5 mln doden
Idi Amin               (°1925) Oeganda        regime: Kannibalisme           3,0 mln doden
Pol Pot                (°1925) Cambodja       regime: Militaire dictatuur    4,0 mln doden
```

1.8. Tenslotte de kers op de taart: **groepeer** alle objecten in bepaalde categorieën en sorteer de namen alfabetisch binnen elke categorie.
Voorbeeld van afdruk (probeer de lay-out zoveel mogelijk te evenaren!) :

```
Alle dictators gegroepeerd per regime:
       Barbarisme          : Atilla de Hun
       Communisme          : Mao Tse-tung
       Despotisme          : Nicolae Ceausescu
       Dictatuur           : Laurent-Desire Kabila
       Fascisme            : Benito Mussolini
       Fundamentalisme     : Ayatollah Khomeini
       Kannibalisme        : Idi Amin
       Kleptocratie        : Ferdinand Marcos, Mobutu Sese-Seko
       Militaire dictatuur : Augusto Pinochet, Francisco Franco, Pol Pot
       Nazisme             : Adolf Hitler
       Stalinisme          : Jozef Stalin, Kim II Sung
```

4. Streams en lambda's

Tenslotte gaan we een aantal eerder geschreven methoden optimaliseren.

4.1. Refactor jouw code in `Demo_4`. Vervang OVERAL de lussen door gebruik te maken van streaming in combinatie met lambda's en/of method references. Er mag dus geen enkele lus meer staan!

4.2. Herschrijf in de klasse `XxxFunctions` alle methoden. Maak NERGENS nog gebruik van een lus; gebruik overal streaming! Codeer zo compact en efficiënt mogelijk.

4.3. Wijzig de methode `remove` in de multiklasse. Waarschijnlijk heb je daar een lus gebruikt; veel te langdradig! Vervang door één statement: een call van de methode `removeIf` in combinatie met een lambda.

4.4. Wijzig de methode `search` in de multiklasse. Waarschijnlijk heb je daar een lus gebruikt; veel te langdradig! Vervang door één stream-statement. (TIP: `orElse`).

Run `Demo_4` opnieuw en controleer de correcte werking.

Zorg ervoor dat je code gesynchroniseerd wordt met je repository op GitLab.